Who Pays for the Parent? Transaction Dependency, Blockspace Allocation, and Confirmation-Delay Redistribution in Bitcoin

Anonymous submission

***Abstract*—Aggregate delay statistics can conceal a consequential redistribution: CPFP may advance a parent-linked package while postponing unrelated transactions, even when cohort mean delay and Gini do not move. We measure this allocation effect using valid signed transactions in Bitcoin Core v31.0 on Regtest. The original 800 matched fixtures cross twelve equal-weight simultaneous transactions, forty fee assignments, five topologies, and four budgets. In the four-slot chain, Core versus scalar raises unrelated restricted waiting by 0.583 blocks and first-block fees by 8,369.5 sat; against ancestor-package selection, both signs reverse. Mean delay and Gini remain unchanged in these comparisons, as confirmation-count invariance predicts. An actual v30.2 template comparison matches v31.0's initial selected set in 751 of 800 cases. A further 780 fixtures vary size and arrival timing and extend five cells to 100 fee seeds. In the exploratory four-slot chain, heterogeneous sizes reverse the ten-seed unrelated-delay contrast to -0.150 blocks (95% interval [-0.300,0.000]), exposing a boundary of the equal-size finding. Independent reconstruction and official-binary replay check the archived blocks. These controlled results identify a local allocation mechanism; they do not estimate Mainnet delays or user welfare.**

***Index Terms*—Bitcoin, cluster mempool, CPFP, blockspace allocation, confirmation delay, reproducibility**

I. Introduction

Two block builders can report the same aggregate delay while confirming different transactions first. When a transaction spends an unconfirmed parent's output, the parent must precede it in a block. A high-fee child can therefore raise its low-fee parent's effective selection priority and move unrelated transactions to later blocks. If all transactions arrive together and each selector confirms the same number in every block, however, the cohort's mean waiting time and Gini remain identical. Aggregate delay statistics can hide who receives priority.

The economic importance of transaction ordering is established. Fee-market models connect congestion and waiting with user bids and mining incentives [1], [2]; mechanism-design research studies which payment rules make those incentives compatible [3], [4]. Historical Bitcoin evidence also shows that client defaults and major intermediaries shape fee practices [5]. Kamada and Noda [6] jointly model dynamic user bidding and miner operation. Queueing studies derive fee-dependent confirmation times [7], [8], and a mempool risk-process model links waiting-time distributions to fee choice [9]. These perspectives explain the incentives around blockspace. Reward-instability and mining-gap models allow fees to alter mining behavior [10], [11]. Our experiment fixes bids and the block schedule to isolate the allocation step: given the same offered fees and transaction weights, which transaction identities obtain scarce space, and what can aggregate delay statistics reveal about that choice?

Child-Pays-For-Parent (CPFP) mechanisms are established practice. Bitcoin Core v31.0 implements cluster mempool: transactions partition into connected components, each linearized into fee-rate chunks that serve as selection units [12], [13]. Mollakhani, Wuille and Guo study dependency-respecting linearization algorithms and evaluate them on realistic and synthetic mempools [14]. We use the pinned v31.0 implementation rather than propose a new algorithm. Our contribution is a controlled downstream allocation benchmark that separates three distinct objects: the priority premium assigned to a selection unit, the displacement of unrelated transactions, and the aggregate confirmation-time distribution.

We construct valid topology interventions by signing new transactions for every graph while matching each label's fee, size, weight, and insertion rank. Three selectors operate on each matched set, and Core generates and validates their blocks. This yields 800 fixtures and 2,400 trajectories, with raw transactions, funding outputs, templates, blocks, and independent reconstruction retained. The resulting benchmark couples an implementation-level comparison with a simple interpretive test: identical per-block confirmation counts must preserve the aggregate burden distribution.

The four-slot chain illustrates why both the benchmark and its comparison rule matter. Relative to scalar selection, Core raises unrelated waiting and first-block fees; relative to ancestor-package selection, both contrasts reverse sign. Aggregate mean waiting and Gini are unchanged in both cases. Priority redistribution therefore has no baseline-free sign, and need not worsen aggregate inequality. Separate replacement trials examine acceptance thresholds as an auxiliary channel distinct from block selection. These findings concern controlled synthetic demand and one node's policy; they do not identify Mainnet miner behavior or user welfare.

Code and paper source:
https://github.com/computational-decision-lab/transaction-dependency-allocation.
Full fixtures and data:
https://huggingface.co/buckets/computational-decision-lab/transaction-dependency-allocation.

II. Mechanism, estimands and identification

A. Transactions and feasible allocation

Let $G = (V, E)$ be the directed acyclic graph of unconfirmed transactions. Transaction i has fee f_i, virtual size v_i, and weight w_i. An edge (p, c) means that c spends an output of p. A feasible selected set must contain every unconfirmed ancestor of each selected child and admit a parent-before-child ordering. For transaction-weight budget W, feasibility requires

$$\sum_i w_i x_i \le W, \qquad x_c \le x_p \quad \forall (p,c) \in E, \qquad x_i \in \{0,1\}. \tag{1}$$

Weight and virtual size follow the Segregated Witness definitions [15]. The study's Taproot key-path transactions have no binding sigop adjustment, so raw and policy-adjusted weights coincide. General measurements must distinguish them.

Core partitions a cluster linearization into chunks. A chunk is a selection unit; an entire cluster need not be atomic. At the mempool's initial snapshot, for transaction i in the Core-returned chunk C_i, define the dependency premium

$$DP_i = \frac{4 \sum_{j \in C_i} f_j}{\sum_{j \in C_i} w_j} - \frac{f_i}{v_i}. \tag{2}$$

All experimental weights are divisible by four, so virtual-size rounding differences vanish. The premium is neither a payment to the parent nor a guarantee of next-block inclusion. The allocator recomputes selection after each block; this snapshot measure captures only the initial state.

B. Confirmation and competing outcomes

For a fixed follow-up horizon $T = 8$ scheduled blocks (4,800 seconds) from each transaction's own arrival, let $F_{m,\mathrm{conf}}(t)$ denote confirmation cumulative incidence under selector m. Define restricted nonconfirmation time as

$$R_m(T) = \int_0^T [1 - F_{m,\mathrm{conf}}(u)]\, du. \tag{3}$$

Here, restricted means truncated at the follow-up horizon. If a_i and c_i are arrival and confirmation times, $B_i = \min(c_i - a_i, T)$; an unconfirmed transaction receives $B_i = T$. The cohort mean equals Eq. 3. For n transactions and $\bar{B} > 0$, we compute $\mathrm{Gini} = \sum_{i,j} |B_i - B_j| / (2n^2 \bar{B})$ and report it alongside confirmation incidence. This standard dispersion index [16] is unchanged by permuting transaction identities. An all-unconfirmed cohort has zero Gini and maximal mean burden, illustrating why inequality alone cannot serve as an efficiency criterion.

The event vocabulary distinguishes confirmation, replacement, expiry, eviction, other removal, unknown removal, and administrative end of observation. Aalen–Johansen estimation retains nonconfirmation removals as competing outcomes [17], [18], rather than treating fee-dependent removals as uninformative censoring. Our selection experiments observe only confirmation and administrative censoring at horizon T; separate replacement trials observe replacement and confirmation. We do not claim to have empirically exercised eviction or expiry effects. Replacement concerns the original transaction identifier, not whether a user's payment intent is eventually satisfied by a successor.

Labels 6–11 are disconnected singletons in every matched topology and form the prespecified unrelated cohort U. The primary displacement contrast is

$$DE_U = R_{\mathrm{cluster},U}(T) - R_{\mathrm{scalar},U}(T). \tag{4}$$

It can be positive, negative, or zero. A separate topology contrast holds the selector fixed and compares regenerated dependent versus independent graphs. The first changes the selection rule on identical signed transactions; the second changes valid transaction dependencies under matched observable demand. They answer different counterfactual questions.

C. Revenue and utilization

For each initial mempool, exhaustive enumeration of up to $2^{12} = 4096$ subsets yields the exact maximum fee $F^* = \max_{x \text{ satisfying (1)}} \sum_i f_i x_i$. We report selected fees F_m, fee capture F_m/F^*, its complement $\mathrm{Gap}_m = 1 - F_m/F^*$, and utilization $\sum_i w_i x_i / W$. With staged arrivals, the optimum uses only transactions already available. This is a first-block revenue benchmark, not a social welfare optimum. Total fees coincide if the entire finite cohort eventually confirms; oversized chunks can prevent that condition under the tested budgets.

D. Why redistribution need not change inequality

Proposition 1 (confirmation-count invariance, standard). Suppose a fixed cohort arrives simultaneously, block times and the observation horizon are common, and two selectors confirm the same number of transactions in each scheduled block. Then their multisets of restricted burdens are identical, hence their aggregate restricted means and Gini coefficients are equal, even when individual confirmation identities differ.

Proof. Each block at time t_b contributes exactly n_b copies of burden t_b to the multiset; transactions not confirmed by T each contribute T. The sequence (n_b, t_b) and cohort size fully determine the burden multiset. Both mean and Gini are symmetric functions of that multiset. □

Equal weights and full utilization provide a transparent example. Consider four equal-size transactions with fee rates 2, 100, 40, and 39 sat/vB, where the second spends the first. A two-slot scalar selector initially takes the unrelated transactions at rates 40 and 39; a package selector can take parent plus child at average rate 51. The unrelated transactions each wait one additional block, while the aggregate confirmation-time multiset remains $\{1, 1, 2, 2\}$ blocks. This analytical illustration does not estimate the frequency of such patterns on Bitcoin.

The proposition constrains empirical interpretation. Under simultaneous demand, a change in aggregate delay dispersion requires a change in confirmation counts or competing outcomes, not merely a reordering of identities. Heterogeneous sizes can change those counts. With staggered arrivals, identical calendar-time counts need not preserve the distribution of time since arrival. Our extensions test both boundaries and retain utilization and confirmation incidence alongside Gini.

III. Executed experimental design

A. Pinned implementation and local policies

The reference is Bitcoin Core tag v31.0, commit `6574cb40869b96b9ffc79c19dc8f4e467d60f321`. We built source with GCC 12.3 and CMake 3.22.1 in Release mode, with wallet enabled and GUI, IPC, and ZMQ disabled. A separately downloaded Linux release binary was checked against its SHA-256 manifest and 13 builder signatures. The archived build manifest records dependency revisions, flags, and binary hashes. This local source build is not claimed to reproduce the Guix release binary byte for byte.

Nodes disable peer networking and bind RPC to loopback. Default cluster limits are retained. Three selectors receive identical signed transactions and transaction-weight budgets:

1) *Scalar:* Repeatedly select the highest f_i/v_i among transactions whose remaining unconfirmed parents have already been selected; skip transactions that do not fit. Ties use lexicographic transaction identifiers.
2) *Ancestor package:* Recompute each candidate's unselected ancestor package, rank by aggregate fee divided by aggregate virtual size, and select the highest-rate fitting package in topological order. Ties use the candidate identifier. Already-selected ancestors are not counted twice.
3) *Core cluster:* Capture the pinned node's `getblocktemplate` transaction list. Record `getmempoolcluster` and `getmempoolfeeratediagram` outputs but do not substitute a reconstructed cluster order for the actual template.

The first two are declared research policies, not reconstructions of historical miners. An exact rational-rate audit of the original 800 fixtures finds no winning-score tie in 9,600 scalar and 8,770 ancestor-package selection rounds. Transaction-identifier tie breaks therefore do not affect those comparator outcomes; this check does not characterize Core's internal tie handling.

Core's block assembler rejects a chunk if reserved plus transaction weight is *greater than or equal to* its configured maximum. We reserve 2,000 weight units and set the maximum to $2000 + W + 1$, making the common transaction budget inclusive at W. This exact boundary avoids confounding a one-unit implementation detail with topology effects. Sigop limits and minimum relay fees are nonbinding in these fixtures.

B. Valid matched graph construction

In the original matrix, every transaction uses one input, six Taproot outputs, version two, zero locktime, an explicitly replaceable sequence number, and one fixed-length Schnorr witness. Each is 1,304 weight units and 326 vB. A selected input is funded either by a matured Regtest coinbase UTXO or by a designated parent output. Output values set the exact fee. Every topology is signed anew, decoded, checked with `testmempoolaccept`, and admitted by Core. Signed bytes are never rewired.

The allocation matrix contains twelve labeled transactions, with labels 0–5 forming the potentially dependent group and 6–11 remaining unrelated. The five graph settings are independent transactions; a two-transaction CPFP pair; a four-transaction chain; a one-parent/five-child star; and a mixed branching component. Forty pseudorandom fee assignments are paired across all graphs and selectors. Label 0 pays 326 sat; label 1 receives the high fee specified by the frozen pseudorandom seed; other labels receive distinct rates drawn from 2–120 sat/vB with small label offsets. The experiment deliberately spans synthetic CPFP incentives and does not fit a Mainnet fee distribution.

For each seed, label-level fees, sizes, weights, and insertion ranks match exactly across graphs. All transactions arrive at a common mock time. Fixed output counts permit legal star and chain variants without changing size. Funding-output values and identifiers change and are retained, but are not modeled utility or user-wealth variables. Every realization passes matching and validity checks before inclusion in the controlled contrast.

Budgets accommodate 2, 4, 6, or 12 transactions per block, yielding 800 graph/seed/budget fixtures and 2,400 selector trajectories. Candidate blocks are generated through Core's `generateblock`. Raw blocks establish inclusion. Branch invalidation restores the exact signed transaction set between selectors; equal-size distinct coinbase scripts avoid reusing an invalidated block hash. Mock time advances in 600-second steps, so delays are controlled block schedules, not sampled Mainnet block intervals.

The two-slot budget, and some four-slot star cases, are stress conditions: a high-rate chunk may exceed the entire block budget and remain excluded at every subsequent step. Default Mainnet block capacity is much larger than a default cluster size, so this specific permanent-exclusion construction is not presented as a typical Mainnet failure mode. The twelve-slot setting is a no-congestion control. Separate 64- and 65-transaction chains test the cluster-count boundary without entering the twelve-transaction causal matrix.

C. Size, arrival, precision, and version extensions

An exploratory follow-up adds 780 fixtures and 2340 selector trajectories: two size assignments, three arrival schedules, four graphs (independent, CPFP, chain, star), two weight budgets, and ten fee seeds. Heterogeneous sizes are exactly 326, 652, and 1,304 vB, obtained with 6, 13, or 28 Taproot outputs and explicit data padding. Labels have fixed size factors $(1, 4, 2, 1, 2, 4, 1, 2, 4, 1, 2, 4)$. Fees match by label across topology and size treatments, so changing size also changes fee rates and offered weight. Budgets remain 4×1304 or 6×1304 WU, not counts of heterogeneous transactions.

All labels arrive at time zero in the simultaneous schedule. In *children-late*, labels 1–5 arrive after the first block; in *two-batch*, labels 3–5 and 9–11 do so. Remaining labels arrive at zero, and no child precedes its parent. Waiting starts at each label's own arrival; a ninth calendar block provides eight-block follow-up for late arrivals. An additional 300 fixtures extend seeds 40–99 for four-unit chain, star, and independent graphs

and six-unit chain and independent graphs. Combined with the original forty seeds, these five cells have 100 seed units each. The ten-seed diagnostic reruns are not added as independent precision samples.

We also replay the original 800 signed sets into the official Core v30.2 binary [19], pinned to commit `4d7d5f6b79d4c11c47e7a828d81296918fd11d4d`. Its checksum and eight builder signatures are verified. Both versions use the same inclusive weight budget; source inspection confirms the same strict capacity boundary. This comparison records actual first-block templates. It does not reconstruct v30 confirmation trajectories or isolate cluster mempool from all other changes between releases.

D. Uncertainty and reproducibility scope

The repeated unit is the fee-assignment seed. We compute paired differences within each seed and use 10,000 paired bootstrap draws for descriptive 95% intervals [20]. Transactions and bootstrap draws do not count as independent replications. Seeds recur across capacities and graphs, so cell estimates are not independent tests. The original surface uses forty seeds, targeted precision cells use 100, and size/arrival diagnostics use ten. Follow-up designs were fixed before their execution but after inspecting the baseline; they are exploratory, without a multiple-testing-adjusted confirmatory or Mainnet sampling claim.

Fresh wallets yield different raw identifiers. Regeneration targets identical label-level fees, sizes, and graphs, with corresponding allocation metrics; captured bytes permit replay of the exact realization. The run retains an initial source snapshot and a recorded orchestration correction that carries mock time across node restarts. Previously completed fixtures were preserved. No treatment configuration was selected or removed in response to an observed outcome.

IV. Results: redistribution and capacity effects

A. Redistribution without aggregate inequality change

The independent-transaction controls produce zero differences in measured allocation outcomes across all three selectors at every capacity. In four-slot chain cases, Core minus scalar unrelated-burden contrast is 0.583 blocks (95% paired-bootstrap interval [0.508, 0.658]), or 350.0 scheduled seconds. First-block fees increase by 8,369.500 sat on average, yet full-cohort mean burden and Gini remain exactly unchanged because each selector still confirms four transactions in each of three blocks. The result is redistribution of which identities wait, as Proposition 1 predicts.

For the child-pays-for-parent pair, the corresponding unrelated contrast is 0.258 blocks [0.208, 0.308]. The mixed graph yields the same aggregate contrasts as the chain for these fee assignments; this does not establish their equivalence under general demand. At six slots, the chain contrast is 0.283 blocks [0.225, 0.342]. Table II and Fig. 1 retain every tested capacity and graph, including null differences. First-block utilization equals one for every policy and cell; capacity stress effects arise in subsequent blocks, not initial weight use.

B. When the capacity constraint changes the distribution

The four-slot star shows an unrelated contrast of 0.546 blocks ([0.429, 0.667]), a full-cohort mean-burden increase of 0.333 blocks, and a Gini change of +0.0184. Its confirmation-incidence contrast is -0.0625. These differences combine identity redistribution with a change in the confirmation-count sequence; some generated star chunks exceed the complete four-slot budget. This is why its aggregate distribution can change while the chain's does not. A high initial dependency premium is insufficient to guarantee inclusion when the selection unit does not fit.

Two-slot cases provide more severe capacity stress. The observed Core minus scalar unrelated means remain nonnegative in these cells, but dependent transactions can also fare worse. In the two-slot star, the parent waits 2.900 more blocks on average under Core despite a positive snapshot dependency premium. A selection unit that cannot fit delivers no priority advantage. This is not evidence that the default Mainnet allocator ordinarily excludes whole clusters indefinitely; the maximum default cluster is far smaller than a default full block.

At twelve slots, all twelve transactions are included in the first block under every selector and graph. Fees and utilization agree, confirmation incidence is one, and restricted-burden Gini is zero. Dependencies can still change the snapshot priority premium but generate no delay contrast when all demand fits. This no-congestion null is part of the result.

C. Topology and algorithm comparisons have different meanings

The released analysis also holds the Core selector fixed and compares each dependent graph with its newly signed independent-graph counterpart. These paired topology contrasts preserve fees and serialization size but do not hold raw identifiers or UTXO funding values fixed. They are distinct from the Core minus scalar contrasts in the main table. For the four-slot star, the Core topology contrast in unrelated burden is -0.133 blocks (interval [-0.263, -0.008]), whereas changing from scalar to Core on the star increases it by 0.546 blocks. The counterfactual therefore changes even the sign of the answer. Against the ancestor comparator on the four-slot chain, the unrelated-burden contrast is -0.042 blocks (interval [-0.079, -0.008]), with 1,456.775 fewer first-block satoshis on average. A revenue improvement in one selected block is neither a higher total fee payment by the fixed cohort nor evidence of greater social efficiency.

D. Actual prior-version templates and optimal-fee gaps

On the original 800 fixtures, v30.2 and v31.0 select identical first-block sets in 751 cases and different sets in 49. In the four-unit chain, the v31.0-minus-v30.2 fee contrast is -1,456.775 sat. Mean optimal-fee gaps are 1.817% for v31.0 and 0.646% for v30.2; in the four-unit star they are 7.208% and 2.367%. At six units, the star instead favors v31.0 by 488.4 sat. These are initial-template results under synthetic budgets, not v30 delay estimates or an isolated causal effect of cluster mempool.

TABLE I
100-SEED PRECISION CELLS: CORE CLUSTER MINUS SCALAR POLICY.

Budget	Graph	n	ΔR_U (blocks)	ΔF (sat)	ΔG	Δ gap (pp)
4	Chain	100	+0.552 [0.508,0.593]	+8,055 [5,822,10,431]	+0.000	-6.9
4	Star	100	+0.560 [0.490,0.630]	+14,074 [10,480,17,811]	+0.010	-11.8
6	Chain	100	+0.310 [0.273,0.347]	+31,009 [27,286,34,750]	+0.000	-18.1
4	Independent	100	+0.000 [0.000,0.000]	+0 [0,0]	+0.000	+0.0
6	Independent	100	+0.000 [0.000,0.000]	+0 [0,0]	+0.000	+0.0

Paired seed-level means and percentile bootstrap intervals; F is first-block fee capture and gap is relative to the exact feasible fee optimum. Five targeted cells combine the original 40 seeds and 60 new seeds. Bootstrap draws are not additional experiments.

In the 100-seed four-slot chain, unrelated waiting rises by +0.552 blocks (95% interval [0.508,0.593]) and first-block fees by +8,055 sat ([5,822,10,431]), while Gini remains unchanged and the Core-minus-scalar optimal-fee-gap contrast is -6.9 percentage points. The four-slot star has a Gini increase of +0.010 and a smaller fee gap; the six-slot chain's unrelated-wait contrast is +0.310 blocks. Results in Table I use 100 independent fee seeds per cell.

Ten-seed size/arrival diagnostics show why the original contrast is conditional. In the four-slot chain, Core-minus-scalar unrelated burden changes from +0.550 blocks with equal sizes and simultaneous arrivals to -0.150 (interval [-0.300,0.000]) with heterogeneous sizes; the full-cohort Gini contrast becomes +0.009. Under two-batch arrivals, the corresponding unrelated contrasts are +0.317 and -0.683 blocks, with Gini contrasts +0.047 and +0.013. These exploratory cells are small, and their estimates vary jointly with transaction weight, fee rate, and each transaction's age at observation. They delimit the equal-size simultaneous result rather than establish a general effect.

E. Replacement acceptance is a separate mechanism

Block selection is not an RBF rule. Three fixed-shape trials compare Core acceptance with two explicitly defined research replacement policies. Let D be direct conflicts, R conflicts plus their removed descendants, and A retained unconfirmed ancestors. The scalar comparator requires replaceability signaling, no new unconfirmed parents, an eviction cap of 100 transactions, a candidate rate no lower than each direct conflict, and

$$f_{\text{new}} \geq \sum_{i \in R} f_i + \lceil 0.1 v_{\text{new}} \rceil. \tag{5}$$

The 0.1 sat/vB incremental relay fee is pinned. The ancestor comparator adds

$$\frac{f_{\text{new}} + \sum_{a \in A} f_a}{v_{\text{new}} + \sum_{a \in A} v_a} > \frac{\sum_{i \in R} f_i}{\sum_{i \in R} v_i}. \tag{6}$$

BIP125 documents historical opt-in replacement signaling and conditions [21]; neither declared comparator is an exact implementation of that specification or an old Core policy. Core acceptance uses its own fee-rate-diagram rules [22]. Candidate fees are searched at integer-satoshi resolution against a fixed mempool. Adjacent rejection/acceptance checks and a monotonicity grid are archived, followed by actual submission, before/after diagrams, eviction sets, and a mined successor block.

In the third fixture, the 326-sat ancestor also has a 20,000-sat sibling child. The stricter declared ancestor comparator can require a larger fee than Core; the other two cases coincide. Thus the experiment does not support a blanket claim that cluster policy increases replacement costs. For original

TABLE II
CORE MINUS SCALAR: DELAY AND ALLOCATION CONTRASTS.

Slots	Graph	ΔR_U	95% CI	ΔU	ΔG
2	CPFP	0.650	[0.583, 0.713]	0.000	0.000
2	Chain	0.667	[0.479, 0.863]	0.000	0.026
2	Star	0.196	[0.062, 0.354]	0.000	0.036
2	Mixed	0.667	[0.479, 0.863]	0.000	0.026
4	CPFP	0.258	[0.208, 0.308]	0.000	0.000
4	Chain	0.583	[0.508, 0.658]	0.000	0.000
4	Star	0.546	[0.429, 0.667]	0.000	0.018
4	Mixed	0.583	[0.508, 0.658]	0.000	0.000
6	CPFP	0.163	[0.125, 0.196]	0.000	0.000
6	Chain	0.283	[0.225, 0.342]	0.000	0.000
6	Star	0.600	[0.546, 0.654]	0.000	0.000
6	Mixed	0.283	[0.225, 0.342]	0.000	0.000
12	CPFP	0.000	[0.000, 0.000]	0.000	0.000
12	Chain	0.000	[0.000, 0.000]	0.000	0.000
12	Star	0.000	[0.000, 0.000]	0.000	0.000
12	Mixed	0.000	[0.000, 0.000]	0.000	0.000

ΔR_U: unrelated restricted delay (blocks); ΔU: first-block utilization; ΔG: full-cohort restricted-burden Gini. Positive differences denote increases under Core. CIs use a paired bootstrap over 40 fee seeds. Two-slot and oversized-chunk four-slot star cases are capacity stress tests.

TABLE III
MINIMUM ACCEPTED REPLACEMENT FEES (SAT).

Conflict setting	Core	Scalar	Ancestor
Single 10,000-sat conflict	10,033	10,033	10,033
Conflict plus 30,000-sat child	40,033	40,033	40,033
Retained low-fee ancestor	10,033	10,033	19,675

Each candidate is 326 vB. Core is the pinned implementation; Scalar and Ancestor use the declared research acceptance rules. These three fixtures are deterministic, not population estimates.

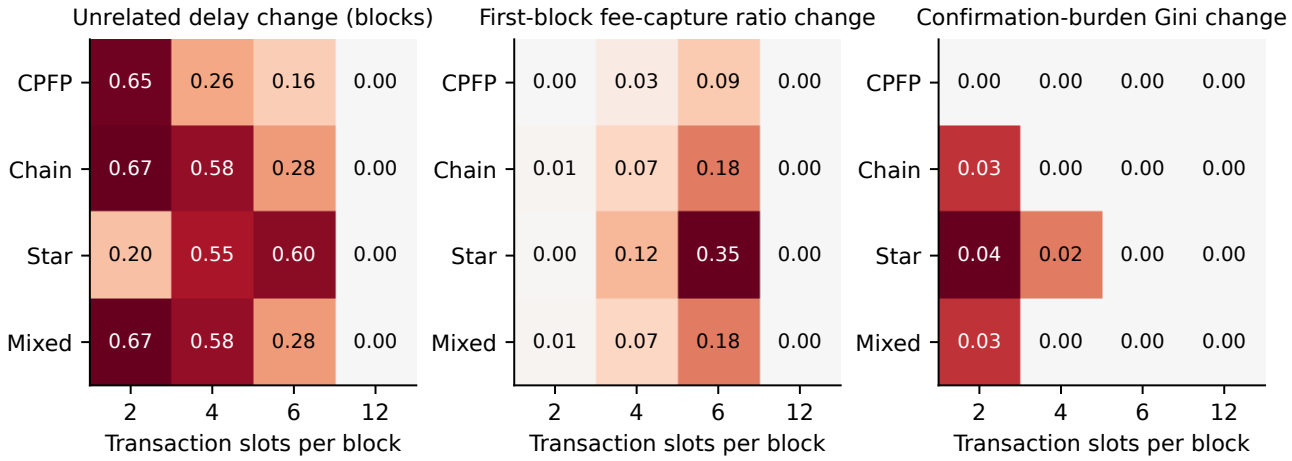


Fig. 1. Core minus scalar contrasts in the original matrix, averaged over forty paired fee assignments. Time differences are in scheduled blocks. A positive delay contrast means later confirmation for the fixed unrelated cohort. Fee capture is selected fees divided by the exact feasible optimum; it and Gini are dimensionless. First-block utilization is identical in every original cell (Table II). The two-slot column and oversized four-slot star chunks are capacity stress tests; the twelve-slot column is a no-congestion control.

transaction identifiers that are replaced, confirmation incidence is zero while their restricted burden remains the full follow-up horizon. Treating those removals as uninformative censoring would change the estimand.

F. Policy boundary and independent verification

The 64-transaction connected fixture is admitted. In the attempted 65-transaction chain, the first 64 transactions are admitted and the last is rejected with `too-large-cluster`. The rejection is recorded as a policy outcome, not a censored confirmation observation.

The standalone parser imports no Core or generator code. It computes transaction and witness identifiers from raw bytes, derives fees from funding prevouts, calculates weights, extracts graph edges, identifies connected components, aggregates cluster and chunk totals, and verifies fee-rate-diagram concavity. It parses saved blocks, verifies transaction Merkle roots and scheduled timestamps, and reconstructs selected sets and terminal events. A second selector implementation uses fixed-point ancestor closure rather than the generator's recursive package construction. Exhaustive feasible-fee optima are cross-checked with a separate binary mixed-integer linear program solver. Direct event fractions, arithmetic means, and pairwise-distance Gini values cross-check the outcome estimator. Eight deliberate corruptions are rejected: altered fee, truncated transaction, changed signature witness, altered chunk weight, altered delay, duplicate terminal event, early censoring passed to a complete-cohort Gini calculation, and a damaged Merkle root. A closed-form three-outcome example checks competing-risk accounting independently of the selection experiment.

Structural schema checks pass all 800 fixtures. Independent HiGHS binary optimization agrees with exhaustive enumeration on all 800 initial fee optima. A pinned offline container reproduces the reconstructed numerical rows. Wallet-free replay into the separately signed official release binary accepts 9,600 raw transactions and reproduces all 800 initial selected sets with 800 exact order matches. This binary comparison covers the first candidate of every fixture; the standalone parser checks all retained trajectory blocks. Relevant upstream cluster, mempool, miner, RBF, and transaction-graph test suites pass 18 cases and 560,880 assertions; 718 other test cases were skipped. These checks establish captured-input agreement, not a proof of all Core behavior or an independent-laboratory replication.

V. Related Work, Interpretation and Limits

The contribution lies between fee-market economics and transaction-graph algorithms. Models of fees and congestion explain the incentives surrounding confirmation [1], [2], [6]; fee-mechanism design evaluates payment rules and strategic behavior [3], [4]. Client-policy evidence shows why concrete implementation choices also matter [5]. Holding bids and block times fixed lets the present design isolate an allocation mechanism, without inferring equilibrium responses from a change in priority. The comparison with linearization research [14] is equally specific: an algorithm's fee-rate ordering does not, by itself, characterize which unrelated transactions wait or whether the cohort's delay dispersion changes. Matched signed graphs, explicit comparators, and confirmation-count invariance make those downstream outcomes separately testable.

Several limits determine the strength of our conclusions. First, twelve-transaction synthetic cohorts, fixed fees, a fixed size assignment, and two staged schedules do not reproduce live traffic or strategic bidding. The size/arrival diagnostics have only ten fee seeds and deliberately stressed weight budgets. Second, candidate templates describe one node's local policy; published mining blocks and network propagation are different observations [23], [24], and local policy is distinct from consensus security [25]. Third, oversized chunks can be excluded at the tested budgets; these tail outcomes do not represent typical full blocks. Fourth, transaction Gini is not

inequality among identified users, and a replaced identifier is not a user's payment intent. Finally, the eight-block horizon truncates waiting; infinite-horizon outcomes and v30 delay trajectories require additional designs.

Independent replay establishes agreement with the specified binaries on captured inputs and reconstructs the numerical claims. It cannot prove external economic validity, consistent with the distinction between simulation verification and validation [26]. Fee capture is not miner profit without accounting for other costs, and waiting time is not a welfare loss without defensible time values and transaction purposes. No Mainnet event history or multi-node observation has been collected for this study. That optional extension has a separate evidence gate and is not silently replaced by Regtest claims.

VI. Conclusion

Dependency-aware selection redistributes confirmation priority, but the measured direction depends on the comparator and on size and arrival conditions. In 100-seed equal-size chain cells, unrelated waits rise without a Gini change; small heterogeneous-size diagnostics can reverse that contrast. The evidence remains a single-node, twelve-transaction Regtest benchmark with synthetic fees and constrained budgets. Future work should compare these boundaries with multi-node Mainnet observations and model user bidding before drawing welfare conclusions.

Reproducibility

Code, paper source, compact results, signed transaction fixtures, funding blocks, RPC captures, source and binary hashes, configurations, comparators, optimization checks, and reproduction instructions are available through the linked public research repository and data archive. A digest-pinned Python container checks offline numerical reconstruction. This document release was frozen at 2026-11-04 09:24:29 CST (UTC+08:00); that timestamp is an artifact identifier, not a Mainnet observation window. Experimental statements are tied to the archived execution and verification artifacts.

References

[1] D. Easley, M. O'Hara, and S. Basu, "From mining to markets: The evolution of Bitcoin transaction fees," *Journal of Financial Economics*, vol. 134, no. 1, pp. 91–109, 2019.

[2] G. Huberman, J. D. Leshno, and C. Moallemi, "Monopoly without a monopolist: An economic analysis of the Bitcoin payment system," *The Review of Economic Studies*, vol. 88, no. 6, pp. 3011–3040, 2021.

[3] T. Roughgarden, "Transaction fee mechanism design," *Journal of the ACM*, vol. 71, no. 4, pp. 1–25, 2024.

[4] S. Basu, D. Easley, M. O'Hara, and E. G. Sirer, "StableFees: A predictable fee market for cryptocurrencies," *Management Science*, vol. 69, no. 11, pp. 6508–6524, 2023.

[5] M. Möser and R. Böhme, "Trends, tips, tolls: A longitudinal study of Bitcoin transaction fees," in *Financial Cryptography and Data Security, FC 2015 Workshops*, ser. Lecture Notes in Computer Science, vol. 8976, 2015, pp. 19–33.

[6] Y. Kamada and S. Noda, "Dynamic user competition and miner behavior in the Bitcoin market," arXiv preprint, 2025, version 1, 21 February 2025. [Online]. Available: https://arxiv.org/abs/2502.15505

[7] S. Kasahara and J. Kawahara, "Effect of Bitcoin fee on transaction-confirmation process," *Journal of Industrial and Management Optimization*, vol. 15, no. 1, pp. 365–386, 2019.

[8] Y. Kawase and S. Kasahara, "Priority queueing analysis of transaction-confirmation time for Bitcoin," *Journal of Industrial and Management Optimization*, vol. 16, no. 3, pp. 1077–1098, 2020.

[9] R. Gündlach, I. V. Stoepker, S. Kapodistria, and J. A. C. Resing, "A holistic approach for Bitcoin confirmation times & optimal fee selection," arXiv:2402.17474, 2024. [Online]. Available: https://arxiv.org/abs/2402.17474

[10] M. Carlsten, H. Kalodner, S. M. Weinberg, and A. Narayanan, "On the instability of Bitcoin without the block reward," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 154–167.

[11] I. Tsabary and I. Eyal, "The gap game," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 713–728.

[12] Bitcoin Core developers, "Bitcoin Core v31.0," 2026, commit 6574cb40869b96b9ffc79c19dc8f4e467d60f321. [Online]. Available: https://github.com/bitcoin/bitcoin/tree/v31.0

[13] ——, "Mempool design and limits," 2026, version 31.0. [Online]. Available: https://github.com/bitcoin/bitcoin/blob/v31.0/doc/policy/mempool-design.md

[14] A. Mollakhani, P. Wuille, and D. Guo, "Bitcoin mempool linearization," arXiv:2607.23787v2, 2026. [Online]. Available: https://arxiv.org/abs/2607.23787v2

[15] E. Lombrozo, J. Lau, and P. Wuille, "BIP 141: Segregated witness (consensus layer)," 2015. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki

[16] J. L. Gastwirth, "The estimation of the Lorenz curve and Gini index," *The Review of Economics and Statistics*, vol. 54, no. 3, pp. 306–316, 1972.

[17] O. O. Aalen and S. Johansen, "An empirical transition matrix for non-homogeneous Markov chains based on censored observations," *Scandinavian Journal of Statistics*, vol. 5, no. 3, pp. 141–150, 1978.

[18] P. K. Andersen, R. B. Geskus, T. de Witte, and H. Putter, "Competing risks in epidemiology: Possibilities and pitfalls," *International Journal of Epidemiology*, vol. 41, no. 3, pp. 861–870, 2012.

[19] Bitcoin Core developers, "Bitcoin Core v30.2," 2026, commit 4d7d5f6b79d4c11c47e7a828d81296918fd11d4d. [Online]. Available: https://github.com/bitcoin/bitcoin/tree/v30.2

[20] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[21] D. A. Harding and P. Todd, "BIP 125: Opt-in full replace-by-fee signaling," 2015. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0125.mediawiki

[22] Bitcoin Core developers, "Mempool replacements," 2026, version 31.0. [Online]. Available: https://github.com/bitcoin/bitcoin/blob/v31.0/doc/policy/mempool-replacements.md

[23] C. Decker and R. Wattenhofer, "Information propagation in the Bitcoin network," in *2013 IEEE Thirteenth International Conference on Peer-to-Peer Computing*, 2013, pp. 1–10.

[24] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun, "On the security and performance of proof of work blockchains," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 3–16.

[25] J. Bonneau, A. Miller, J. Clark, A. Narayanan, J. A. Kroll, and E. W. Felten, "SoK: Research perspectives and challenges for Bitcoin and cryptocurrencies," in *2015 IEEE Symposium on Security and Privacy*, 2015, pp. 104–121.

[26] R. G. Sargent, "Verification and validation of simulation models," *Journal of Simulation*, vol. 7, no. 1, pp. 12–24, 2013.